

K_AI_PIM — AI-for-EDA & Processing-in-Memory

🔥 ML로 설계 PPA·검증을 자동화하는 AI-for-EDA와, 연산을 메모리 안으로 넣어 memory wall을 깨는 PIM(GDDR6-AiM·HBM-PIM)을 잇는 토픽 · 📁 P3, 지원서연계 · 🧠 내 RTL Sign-off Agent·false violation 20%↓ 경험이 곧 AI-for-EDA, PIM은 데이터 이동 최소화라는 설계 철학의 메모리판

핵심 개념

- **AI-for-EDA**: ML로 칩 설계·검증을 자동화·최적화. ① **PPA 최적화**(강화학습 기반 합성·P&R 파라미터 탐색) — Synopsys DSO.ai(RL로 PPA), Cadence Cerebrus. ② **검증 AI** — Lint/CDC false violation 분류, RTL 디버그 보조(Synopsys VSO.ai/LintAdvisor, Cadence ChipStack AI 류).
- **지원서 연결**: 본인 **RTL Sign-off Agent**(deterministic + RAG/LLM 분기, 신뢰도 스코어링)로 **Pilot false violation 20%↓**, RDC 방법론으로 **87.7%↓**, SFR-to-HW CDC noise **95% 분류** — 곧 AI-for-EDA의 실사례.
- **Memory wall**: 연산 성능은 빠르게 느는데 메모리 대역폭·데이터 이동이 못 따라옴 → AI 워크로드의 진짜 병목은 연산이 아니라 **데이터 이동(에너지·지연)**.
- **PIM(Processing-in-Memory)**: 연산 유닛을 **메모리 안/근처**에 뒀 외부로 데이터를 안 옮기고 처리 → memory wall 완화. 외부 핀보다 **내부 DRAM 대역폭이 훨씬 큼**을 활용.
- **SK하이닉스 GDDR6-AiM**(Accelerator-in-Memory): DRAM bank 안에 MAC 연산기. 1GB, **외부 BW 64GB/s vs 내부 GEMV BW 0.5TB/s(8배)**, all-bank 병렬. 특정 연산 최대 **16배 빠르고 데이터 이동 전력 ~80%↓**.
- **AiMX**: GDDR6-AiM 여러 개를 묶은 **AI 가속 카드**. LLM에서 GPU 대비 처리시간 **10배↓·전력 1/5**.
- **HBM-PIM**: HBM die 안에 연산기를 넣는 개념(메모리 병목이 큰 대형 AI). PIM을 HBM의 초광대역과 결합.
- **HBM↔AI 가속기**: HBM이 GPU에 대역폭 공급(외부 해법), PIM이 memory-bound 연산을 메모리 안에서 처리(내부 해법) → 둘이 함께 memory wall 공략.

예상 면접 Q&A

Q1. AI-for-EDA가 무엇이고 본인 경험과 어떻게 연결되나? (지원서연계)

A. AI-for-EDA는 ML을 칩 설계·검증 과정에 넣어 PPA 최적화나 검증을 자동화하는 흐름입니다. 대표적으로 강화학습으로 합성·P&R 파라미터를 탐색해 PPA를 끌어올리는 **Synopsys DSO.ai, Cadence Cerebrus**가 있고, 검증 쪽에는 Lint/CDC의 false violation을 분류·디버그 보조하는 도구들이 있습니다. 제가 한 일이 바로 이 검증 AI의 실사례입니다. 사업부 표준 RTL Sign-off Flow 위에 **도메인 지식 기반 Sign-off Agent**를 만들어, 구조적으로 판정 가능한 rule은 **deterministic tool DB**로, 설계 의도가 필요한 rule은 **RAG로 지식을 주입한 LLM**으로 분기시키고 근거에 신뢰도 스코어를 매겼습니다. 실제 Pilot에서 **false violation 20% 감소**, RDC 방법론으로 **87.7% 감소**를 얻었습니다. - L, 꼬리질문: 검증에서 AI를 sign-off 대체로 못 쓰는 이유? → static sign-off는 100% 정확성이 전제라, AI는 **대체가 아니라 human effort를 줄이는 보조**(분류·우선순위·근거 제시)에 뒤야 한다. 그래서 신뢰도 스코어링·검증 가능한 근거를 남겼다. - 🧠 설계 브릿지: AI-for-EDA의 핵심은 "결과를 사람이 검증 가능하게" — 이걸 제가 sign-off 방법론에서 지킨 원칙 그대로이고 메모리 R&D의 CAD/Methodology에 그대로 옮길 수 있다.

Q2. Memory wall이란 무엇인가?

A. 프로세서의 연산 성능은 빠르게 향상되는데 메모리의 대역폭·지연은 그만큼 못 따라가서, 시스템 성능이 **메모리에서 막히는** 현상입니다. 특히 LLM 추론처럼 가중치를 대량으로 읽어 공급하는 연산은 연산량보다 **메모리에서 데이터를 가져오는 대역폭과, 그 데이터를 옮기는 데 드는 에너지**가 진짜 병목입니다. 그래서 해법이 두 갈래입니다 — 메모리를 연산기 가까이 붙여 대역폭을 키우거나(HBM), 아예 연산을 메모리 안으로 넣어 데이터 이동을 없애거나(PIM)입니다. - L, 꼬리질문: HBM으로 대역폭을 키워도 남는 한계는? → 외부로 데이터를 옮기는 행위 자체의 에너지·지연이 남는다. 이걸 줄이는 게 PIM. - 🧠 설계 브릿지: 데이터 이동 최소화는 캐시 계층·NoC·데이터 locality 최적화와 같은 사고 — memory wall은 컴퓨트-메모리 인터페이스 설계 문제다.

Q3. PIM이 왜 memory wall을 해결하나?

A. 핵심은 **메모리 내부 대역폭이 외부 핀 대역폭보다 훨씬 크다**는 점입니다. DRAM은 내부적으로 수많은 bank가 동시에 열려 엄청난 데이터를 다루지만, 외부로 나가는 핀은 그 일부만 빼낼 수 있습니다. PIM은 연산기를 **DRAM bank 안/근처**에 뒀서, 데이터를 칩 밖으로 옮기지 않고 이 풍부한 내부 대역폭으로 바로 계산합니다. 그 결과 memory-bound 연산(예: LLM의 GEMV)에서 큰 속도 향상과 함께, 데이터 이동이 줄어 **전력이 크게 절감**됩니다. SK하이닉스 GDDR6-AiM은 외부 64GB/s 대비 **내부 GEMV 대역폭 0.5TB/s(약 8배)**를 활용합니다. - L, 꼬리질문: 그럼 모든 연산을 PIM으로? → 아니다. PIM은 **memory-bound·단순 반복(GEMV, 벡터연산)**에 유리하고, compute-bound·복잡 제어는 GPU가 낫다. 둘을 나눠 맡긴다. - 🧠 설계 브릿지: "연산을 데이터 옆으로 옮긴다"는 near-data computing — 데이터패스 locality·dataflow 설계와 같은 원리.

Q4. SK하이닉스의 PIM 제품(GDDR6-AiM·AiMX)을 설명하라.

A. **GDDR6-AiM**(Accelerator-in-Memory)은 GDDR6 DRAM의 각 bank 안에 MAC 연산기를 넣은 PIM으로, all-bank 병렬 동작으로 내부 대역폭을 끌어다 씁니다. 1GB density에 외부 64GB/s 대비 내부 GEMV 대역폭이 0.5TB/s 수준이고, 특정 연산을 일반 구성 대비 **최대 16배 빠르게, 데이터 이동 전력은 약 80% 절감**합니다. **AiMX**는 이 GDDR6-AiM 칩 여러 개를 묶은 **생성형 AI 가속 카드로**, LLM 워크로드에서 GPU 대비 처리시간을 10배가량 줄이고 전력을 1/5로 낮추는 것을 보였습니다. memory-bound한 LLM 추론에 특화된 접근입니다. - L, 꼬리질문: HBM-PIM과의 차이? → GDDR6-AiM은 GDDR 기반 카드형, HBM-PIM은 HBM 적층 안에 연산을 넣어 초광대역과 결합하는 방향. 타깃 시스템·대역폭 규모가 다르다. - 🧠 설계 브릿지: AiM의 bank 내 MAC·명령 디코딩·all-bank 제어는 전형적 디지털 RTL — 메모리 안에 들어간 작은 데이터패스 설계다.

Q5. HBM과 PIM, AI 가속기는 시스템에서 어떻게 함께 가나?

A. 둘은 memory wall을 양쪽에서 공략하는 상보적 해법입니다. **HBM**은 GPU die 옆 2.5D 인터포저에서 TB/s급 대역폭을 공급하는 **외부 해법**으로, 가속기에 데이터를 빠르게 먹입니다. **PIM**은 memory-bound 연산을 아예 메모리 안에서 처리해 데이터 이동을 없애는 **내부 해법**입니다. 시스템은 compute-bound 연산은 GPU+HBM에, memory-bound 연산(GEMV, attention의 일부)은 PIM에 나눠 맡겨 전체 효율을 높입니다. SK하이닉스는 HBM 시장 1위(~52%)이면서 GDDR6-AiM/AiMX, HBM-PIM으로 두 축을 모두 갖춘 **AI 메모리 종합 공급사**를 지향합니다. - L, 꼬리질문: 신입 설계자로서 여기서 기여할 부분? → PIM/base die의 연산·제어 로직 RTL 설계·검증, AI 기반 검증 자동화로 그 개발 TAT를 줄이는 일. - 🧠 설계 브릿지: 제 강점인 디지털 설계·CDC·STA·검증 자동화가 PIM 로직과 HBM base die 같은 "메모리 속 디지털"에 직접 닿고, AI-for-EDA 경험으로 그 개발을 가속할 수 있다.

30초 암기 요약

- **AI-for-EDA** = ML로 PPA(DSO.ai·Cerebrus)·검증(Lint/CDC AI) 자동화. 내 **Sign-off Agent**로 false violation 20%↓, RDC 87.7%↓ = 실사례(지원서연계).
- AI는 sign-off **대체 아닌 보조** — 신뢰도 스코어·검증 가능한 근거가 핵심.
- **Memory wall** = 연산↑ vs 데이터 이동(대역폭·에너지)↓ → 진짜 병목.
- **PIM** = 연산을 메모리 안으로 → **내부 대역폭 >> 외부 핀** 활용, memory-bound 연산 가속·전력 절감.
- **GDDR6-AiM**: bank 내 MAC, 내부 GEMV 0.5TB/s(외부의 8배), 특정 연산 16배·전력 80%↓. **AiMX=AiM** 카드, LLM GPU 대비 10배·전력 1/5.
- **HBM(외부 대역폭) + PIM(내부 연산)**이 상보적으로 memory wall 공략. SK하이닉스=AI 메모리 종합 공급사.